{EPITECH}

CONVENTION DE NOMMAGE DES COMMITS

web@cadémie

BY EPITECH





EXEMPLES DE COMMIT A NE PAS FAIRE

Ils ne veulent rien dire !

Ne sont pas clairs du tout !

Il y a une façon de nommer qu'il faut respecter.

Un commit est sensé « raconter l'histoire du projet », c'est l'historique du projet

Imaginez que vous revenez sur ce commit après 6 mois. Le message ne vous dira rien.

FORMAT :

<TYPE> (<scope>) :
<DESCRIPTION>

Type : catégorie

Scope : champ optionnel qui décrit la partie du projet affecté

Sujet : résumé concis du changement

TYPE DE BON COMMIT

Un bon message de commit permet de savoir ce qui a été réalisé et pourquoi !

Permet de décrire les fonctionnalités, correctifs et modifications importantes.

Sert aussi au suivi de bugs=> plus facile de localiser les commits contenant des bugs.

NOMMAGE DES MESSAGES DE COMMIT

Eléments structurels

- Feat ou Add : Introduction nouvelle fonctionnalité
- Fix : Correction de bug
- Docs : Documentation (readme, guide, …)
- Test : Ajout/modification de tests
- Refactor : Changement dans le code qui n'impacte pas le code (refactoring)
- Style : Modification de style (identation, mise en forme, renommage d'une variable, …)
- Delete ou Remove : Suppression de fichier, code, …

NOMMAGE DES MESSAGES DE COMMIT

Convention

- Séparer les features qui ont été crée (ex : create user = un commit, update = un commit, delete = un commit)
- On utilise l'impératif présent : add, change, update, remove et pas added, changed, updated, removed…
- Nommer les branches en fonction de ce qui est fait = "fix/update user"
- Limité à 50 caractères
- De préférence, commit en anglais !
- On évite les majuscules et n'ajoute pas de point à la fin.
- Des emojis peuvent être ajoutés (doit avoir du sens)

DOCUMENTATIONS TECHNIQUES

Pour aller plus loin, voici des ressources utiles :

- Guide officiel des commits conventionnels : https://www.conventionalcommits.org

- Documentation git : https://git-scm.com/doc

DES QUESTIONS ?

